Artificial Intelligence Application Development Practical

Smart Movie Recommendation System Using Machine Learning

Student Name: Vijay Bisht

Roll Number: T004

Class: B.Sc. Information Technology

Division: A

Subject: Artificial Intelligence Application Development Practical

Academic Year: 2026–2027

College Name: SHETH L.U.J & SIR MV COLLAGE

Department Name: IT

Faculty Name: Ms. Chetana Tanawade

1. INTRODUCTION

The Smart Movie Recommendation System Using Machine Learning is an intelligent application designed to recommend movies to users based on their interests and preferences. With a large number of movies available on digital platforms, users may find it difficult to decide what to watch. A recommendation system helps solve this problem by identifying movies that are similar to the movie selected by the user.

This project uses Machine Learning techniques such as Content-Based Filtering, TF-IDF Vectorization, and Cosine Similarity to analyze movie information such as title, genre, and description. The system converts movie information into numerical features and calculates the similarity between movies. Based on the similarity scores, the system recommends the most relevant movies.

The application is developed using Python, Pandas, Scikit-learn, and Streamlit. It provides a simple interface where users can select a movie and receive a list of similar movie recommendations along with their similarity scores.

The main purpose of this project is to demonstrate how Machine Learning can be used to provide personalized and intelligent movie recommendations in an easy-to-use application.

2. PROBLEM STATEMENT

With the increasing number of movies available on streaming platforms and digital services, users often find it difficult to choose a movie that matches their interests. Searching through a large collection of movies can be time-consuming and may not always provide relevant results.

The problem is to develop a Smart Movie Recommendation System that can analyze movie information such as genre and description and recommend movies that are similar to a movie selected by the user.

The system uses Machine Learning techniques such as Content-Based Filtering, TF-IDF Vectorization, and Cosine Similarity to calculate the similarity between movies and generate relevant recommendations. The system should provide the recommendations through a simple and user-friendly interface.

3. OBJECTIVES

The main objectives of the Smart Movie Recommendation System Using Machine Learning are:

1. To develop a smart recommendation system that suggests movies based on the user's selected movie.
2. To analyze movie information such as title, genre, and description.
3. To implement Content-Based Filtering for generating relevant movie recommendations.
4. To use TF-IDF Vectorization to convert movie information into numerical features.
5. To calculate movie similarity using the Cosine Similarity technique.
6. To recommend the most similar movies based on their similarity scores.
7. To display the recommendations through a simple and user-friendly Streamlit interface.
8. To demonstrate the practical application of Machine Learning in developing an intelligent movie recommendation system.

4. TECHNOLOGIES USED

The following technologies and tools are used to develop the Smart Movie Recommendation System Using Machine Learning:

1. Python – Used as the main programming language for developing the recommendation system.
2. Pandas – Used for creating, storing, and processing the movie dataset.
3. Scikit-learn – Used to implement Machine Learning techniques such as TF-IDF Vectorization and Cosine Similarity.
4. TF-IDF Vectorization – Converts movie genres and descriptions into numerical feature vectors.

5. Cosine Similarity – Measures the similarity between movies and helps identify the most relevant recommendations.
6. Streamlit – Used to create the interactive and user-friendly web interface for selecting movies and displaying recommendations.
7. VS Code – Used as the development environment for writing and running the Python application.

5. DATASET

The Smart Movie Recommendation System uses a sample movie dataset containing information about different movies. The dataset is created specifically for this project and is stored as a Pandas DataFrame in the Python program.

Dataset Attributes:

Attribute	Description
Title	Name of the movie
Genre	Genre or category of the movie, such as Action, Adventure, Comedy, Fantasy, etc.
Description	Short description containing important information about the movie and its content

The dataset contains 20 sample movies, including movies from popular superhero and action categories. The Genre and Description fields are combined to create a Features column.

These features are then processed using TF-IDF Vectorization to convert the text into numerical values. Cosine Similarity is applied to these values to determine how similar one movie is to another.

Dataset Processing Flow

Movie Dataset → Data Cleaning → Feature Combination → TF-IDF Vectorization → Cosine Similarity → Movie Recommendations

The dataset is used for educational purposes to demonstrate how a Machine Learning-based movie recommendation system works.

6. METHODOLOGY/WORKING

The Smart Movie Recommendation System works by analyzing movie information and finding movies that are similar to the movie selected by the user. The complete methodology is as follows:

Step 1: Data Collection

A sample movie dataset is created containing Title, Genre, and Description of 20 movies.

Step 2: Data Preparation

The movie data is stored in a Pandas DataFrame. The Genre and Description of each movie are combined to create a single Features column.

Step 3: TF-IDF Vectorization

The TF-IDF (Term Frequency–Inverse Document Frequency) technique converts the text in the Features column into numerical vectors. This allows the system to compare the important words present in different movies.

Step 4: Calculate Cosine Similarity

Cosine Similarity is applied to the TF-IDF vectors to measure the similarity between every pair of movies. A higher similarity score indicates that two movies have more similar content.

Step 5: Select a Movie

The user selects a movie from the Streamlit interface using a dropdown menu.

Step 6: Generate Recommendations

The system finds the selected movie in the similarity matrix and compares its similarity scores with other movies. The movies with the highest similarity scores are selected as recommendations.

Step 7: Display Results

The recommended movies are displayed along with their genre and similarity percentage. A similarity chart can also be displayed to provide a visual comparison.

Overall Working

Movie Dataset → Feature Combination → TF-IDF Vectorization → Cosine Similarity → Select Movie → Find Similar Movies → Display Recommendations

This methodology allows the system to provide content-based movie recommendations without requiring information about other users.

7. ALGORITHM USED

Smart Movie Recommendation System uses Content-Based Filtering with TF-IDF Vectorization and Cosine Similarity to recommend movies.

1. Content-Based Filtering

Content-Based Filtering recommends movies based on the features and content of the movies. In this project, the system uses the Genre and Description of movies to identify movies with similar content.

2. TF-IDF Vectorization

TF-IDF (Term Frequency–Inverse Document Frequency) converts the text information of movies into numerical vectors.

- It identifies important words in the movie descriptions and genres.
- Common words are given less importance.
- Important words receive higher weights.
- These numerical vectors are used for comparing movies.

3. Cosine Similarity

Cosine Similarity measures the similarity between the TF-IDF vectors of two movies.

- A higher similarity score means the movies have more similar content.
- The system calculates similarity between the selected movie and all other movies.
- Movies with the highest similarity scores are recommended.

Algorithm Steps

1. Store the movie dataset.
2. Combine Genre and Description into a Features column.
3. Apply TF-IDF Vectorization to the Features column.
4. Calculate Cosine Similarity between all movies.
5. Allow the user to select a movie.
6. Find the selected movie's similarity scores.
7. Sort the movies according to their similarity scores.
8. Select the top similar movies.
9. Display the recommended movies with their similarity scores.

Algorithm Flow:

Movie Data → Feature Extraction → TF-IDF → Cosine Similarity → Similarity Ranking → Top Movies → Recommendations

8. IMPLEMENTATION

Smart Movie Recommendation System is implemented using Python, Pandas, Scikit-learn, and Streamlit. The system uses Content-Based Filtering, TF-IDF Vectorization, and Cosine Similarity to generate movie recommendations.

CODE:

```
import streamlit as st
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
from sklearn.metrics.pairwise import cosine_similarity

# ============================================================
# PAGE CONFIGURATION
# ============================================================

st.set_page_config(
    page_title="AI Movie Recommendation System",
    page_icon="🎬",
    layout="wide"
)

# ============================================================
# TITLE
# ============================================================

st.title("🎬 AI-Based Movie Recommendation System")

st.write(
    "An AI-based movie recommendation system that recommends "
    "similar movies using Content-Based Filtering, TF-IDF "
    "and Cosine Similarity."
)

st.write("A Group Project Made By Ishita & Subhashree")

# ============================================================
# SAMPLE MOVIE DATASET
# ============================================================

data = {

    "Title": [
        "The Avengers",
        "Avengers Age of Ultron",
        "Iron Man",
        "Iron Man 2",
        "Captain America The First Avenger",
        "Captain America The Winter Soldier",
        "Thor",
        "Thor The Dark World",
        "Black Panther",
        "Doctor Strange",
        "Spider Man",
        "Spider Man Homecoming",
        "Guardians of the Galaxy",
        "Guardians of the Galaxy Vol 2",
        "Ant Man",
        "Justice League",
```

```
    "Batman Begins",
    "The Dark Knight",
    "Man of Steel",
    "Wonder Woman"
],

"Genre": [
    "Action Adventure Science Fiction",
    "Action Adventure Science Fiction",
    "Action Science Fiction",
    "Action Science Fiction",
    "Action Adventure",
    "Action Adventure Thriller",
    "Action Adventure Fantasy",
    "Action Adventure Fantasy",
    "Action Adventure Science Fiction",
    "Action Adventure Fantasy",
    "Action Adventure Science Fiction",
    "Action Adventure Science Fiction Comedy",
    "Action Adventure Science Fiction Comedy",
    "Action Adventure Science Fiction Comedy",
    "Action Adventure Science Fiction Comedy",
    "Action Adventure Fantasy",
    "Action Crime Drama",
    "Action Crime Drama Thriller",
    "Action Adventure Science Fiction",
    "Action Adventure Fantasy"
],

"Description": [
    "superheroes avengers save the world action team",
    "avengers superheroes fight powerful enemy action team",
    "iron man superhero technology action marvel",
    "iron man technology superhero action marvel",
    "captain america superhero war action marvel",
    "captain america winter soldier action thriller marvel",
    "thor superhero god fantasy action marvel",
    "thor fantasy superhero action adventure marvel",
    "black panther superhero wakanda action marvel",
    "doctor strange magic superhero fantasy marvel",
    "spider man superhero action adventure marvel",
    "spider man young superhero school action marvel",
    "guardians heroes space adventure superhero marvel",
    "guardians heroes space adventure comedy marvel",
    "ant man superhero technology comedy marvel",
    "superheroes justice league action adventure dc",
    "batman hero crime action drama dc",
    "batman superhero crime thriller action dc",
    "superman superhero action science fiction dc",
```

```
        "wonder woman superhero action fantasy dc"
    ]
}

df = pd.DataFrame(data)

# ============================================================
# CREATE COMBINED FEATURES
# ============================================================

df["Features"] = (
    df["Genre"] + " " +
    df["Description"]
)

# ============================================================
# TF-IDF VECTORIZATION
# ============================================================

vectorizer = TfidfVectorizer(
    stop_words="english"
)

feature_matrix = vectorizer.fit_transform(
    df["Features"]
)

# ============================================================
# COSINE SIMILARITY
# ============================================================

similarity_matrix = cosine_similarity(
    feature_matrix
)

# ============================================================
# RECOMMENDATION FUNCTION
# ============================================================

def recommend_movies(movie_name, number_of_movies=5):

    movie_index = df[
        df["Title"] == movie_name
    ].index[0]

    similarity_scores = list(
        enumerate(similarity_matrix[movie_index])
    )
```

```
    similarity_scores = sorted(
        similarity_scores,
        key=lambda x: x[1],
        reverse=True
    )

    recommendations = []

    for index, score in similarity_scores[1:number_of_movies + 1]:

        recommendations.append({
            "Movie": df.iloc[index]["Title"],
            "Genre": df.iloc[index]["Genre"],
            "Similarity": score
        })

    return pd.DataFrame(recommendations)

# ========================================================
# USER INPUT
# ========================================================

st.header("🎥 Select a Movie")

selected_movie = st.selectbox(
    "Choose a movie you like:",
    df["Title"]
)

number_of_movies = st.slider(
    "Number of recommendations",
    min_value=3,
    max_value=10,
    value=5
)

# ========================================================
# RECOMMENDATION BUTTON
# ========================================================

if st.button("🔍 Recommend Movies"):

    recommendations = recommend_movies(
        selected_movie,
        number_of_movies
    )

    st.subheader("🎬 Recommended Movies")
```

```
    for i, row in recommendations.iterrows():

        st.write(
            f"### {i + 1}. {row['Movie']}"
        )

        st.write(
            f"**Genre:** {row['Genre']}"
        )

        st.write(
            f"**Similarity Score:** "
            f"{row['Similarity'] * 100:.2f}%"
        )

        st.progress(
            float(row["Similarity"])
        )

        st.divider()

# ========================================================
# RECOMMENDATION CHART
# ========================================================

if st.button("📊 Show Recommendation Chart"):

    recommendations = recommend_movies(
        selected_movie,
        number_of_movies
    )

    chart_data = recommendations.set_index(
        "Movie"
    )["Similarity"]

    st.subheader(
        "📊 Movie Similarity Chart"
    )

    st.bar_chart(
        chart_data
    )
# ========================================================
# DATASET PREVIEW
# ========================================================
st.header("📁 Movie Dataset")
with st.expander("View Movie Dataset"):
    st.dataframe(
```

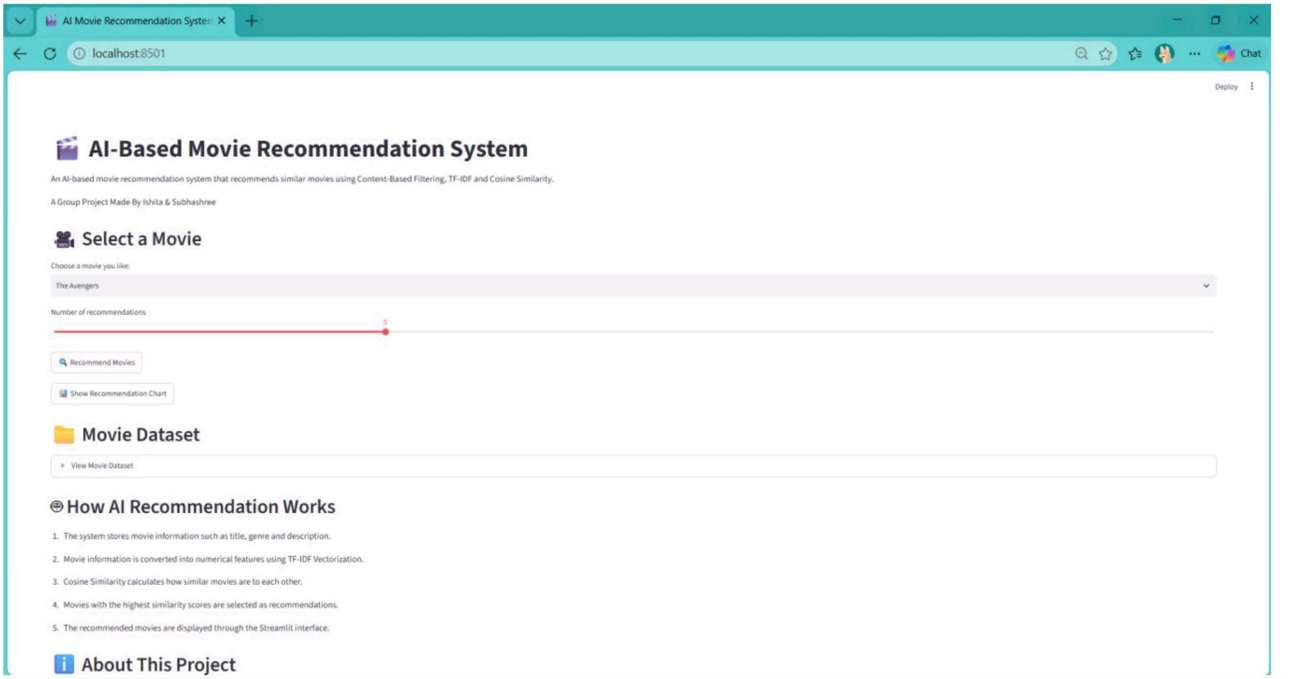
```
        df[
            ["Title", "Genre", "Description"]
        ],
        use_container_width=True
    )
# ========================================================
# HOW THE SYSTEM WORKS
# ========================================================
st.header("🤖 How AI Recommendation Works")
st.write(
    """
    1. The system stores movie information such as title,
       genre and description.
    2. Movie information is converted into numerical
       features using TF-IDF Vectorization.
    3. Cosine Similarity calculates how similar movies
       are to each other.
    4. Movies with the highest similarity scores are
       selected as recommendations.
    5. The recommended movies are displayed through
       the Streamlit interface.
    """
)
# ========================================================
# ABOUT PROJECT
# ========================================================
st.header("ℹ️ About This Project")
st.write(
    """
    This project uses Content-Based Filtering to recommend
    movies based on their similarity.
    TF-IDF is used to convert movie descriptions and genres
    into numerical features, while Cosine Similarity is used
    to measure similarity between movies. """
)
# ========================================================
# DISCLAIMER
# ========================================================st.info(
    "ℹ️ This project is developed for educational purposes "
    "to demonstrate Artificial Intelligence and Machine "
    "Learning concepts."
)
```

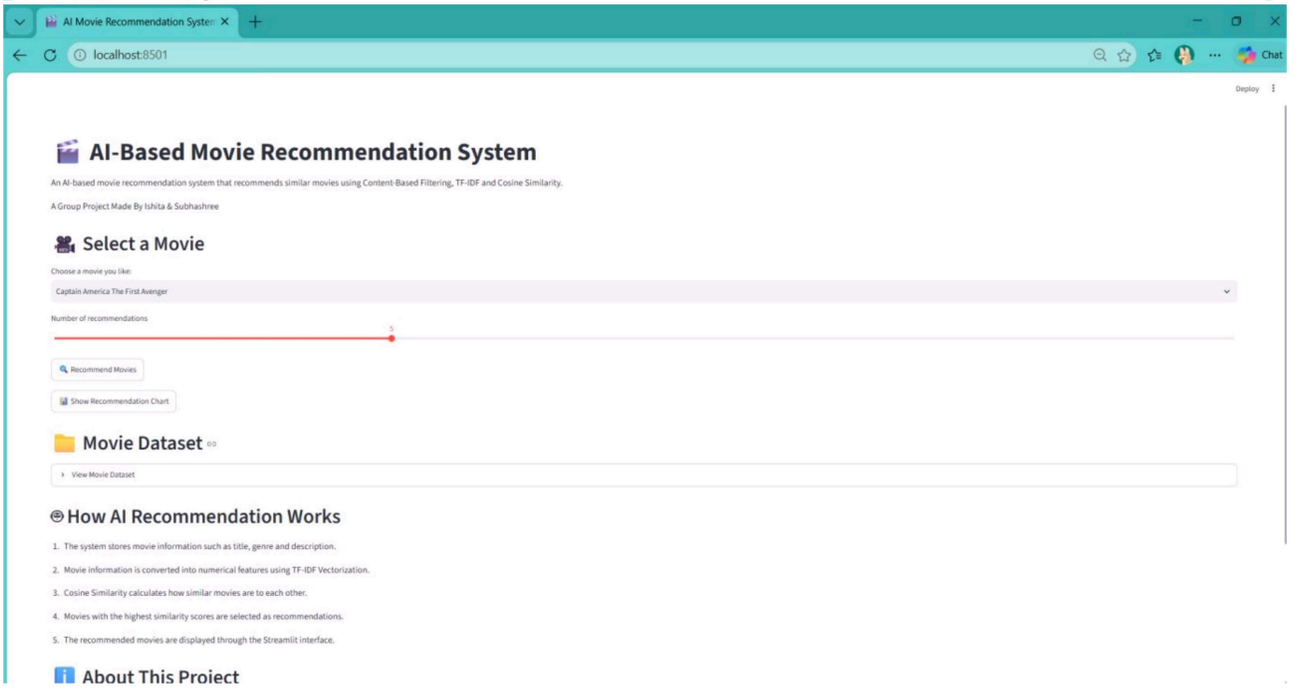
CODE EXPLAINATION:

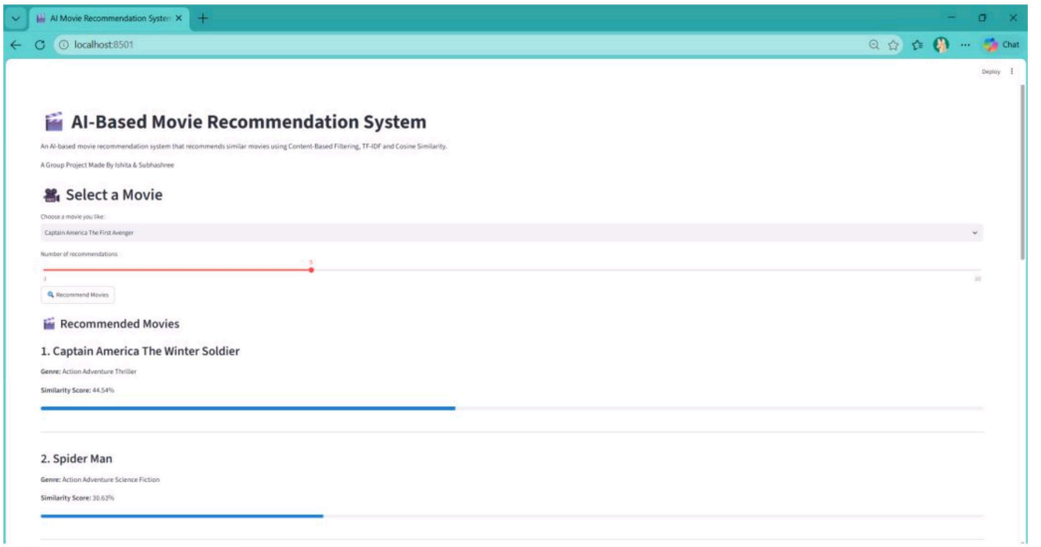
The Smart Movie Recommendation System is developed using Python, Pandas, Scikit-learn, and Streamlit. First, the required libraries are imported and a sample movie dataset containing Title, Genre, and Description is created using Pandas. The Genre and Description are combined into a single Features column, which provides the information required for recommendation. The TF-IDF Vectorizer converts these textual features into numerical vectors, and Cosine Similarity calculates the similarity between all movies. A recommendation function takes the movie selected by the user, finds its similarity scores, sorts the movies from highest to lowest similarity, and selects the top similar

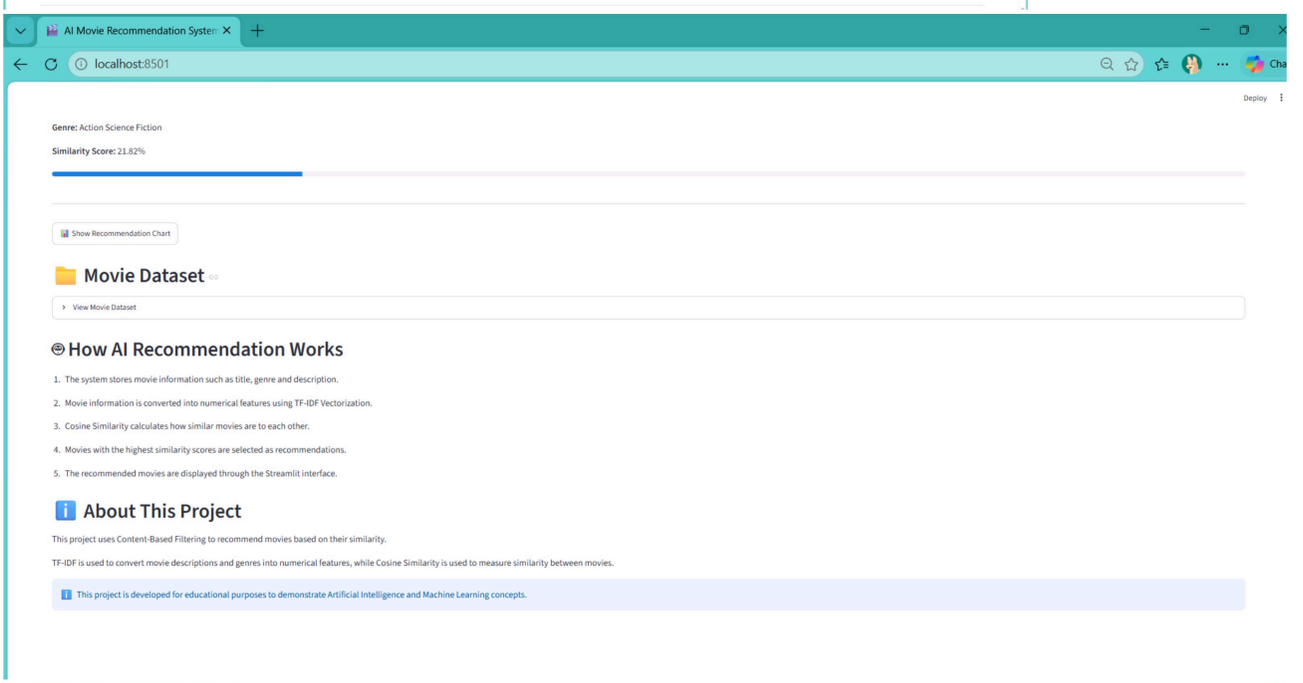
movies while excluding the selected movie itself. The Streamlit interface allows the user to select a movie and choose the number of recommendations. When the Recommend Movies button is clicked, the system displays the recommended movies along with their genres, similarity scores, and progress bars. A similarity bar chart can also be displayed for comparison. The dataset can be viewed through the Streamlit interface using an expandable section. Thus, the complete code follows the process Movie Dataset → Feature Creation → TF-IDF → Cosine Similarity → Similarity Ranking → Movie Recommendations → Streamlit Output.

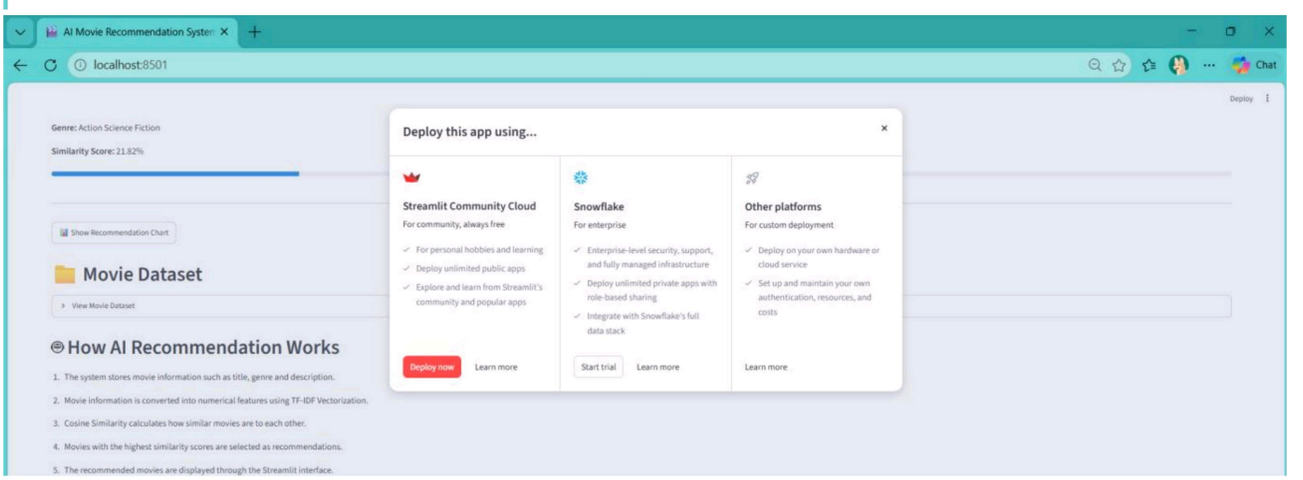
OUTPUT:

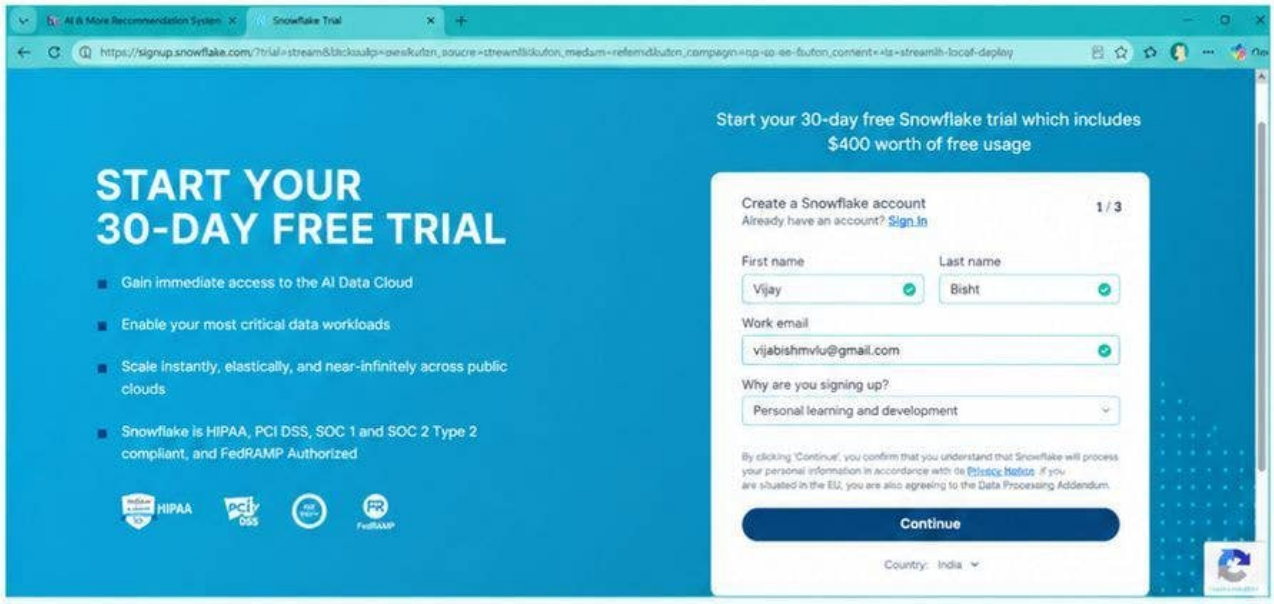


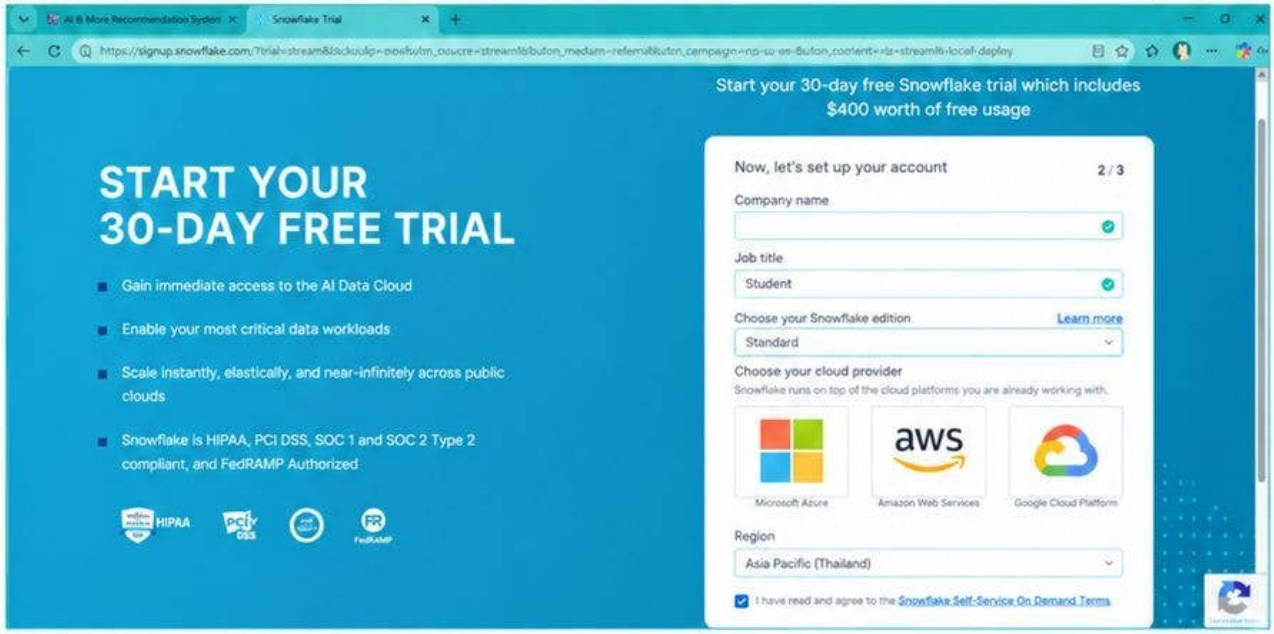


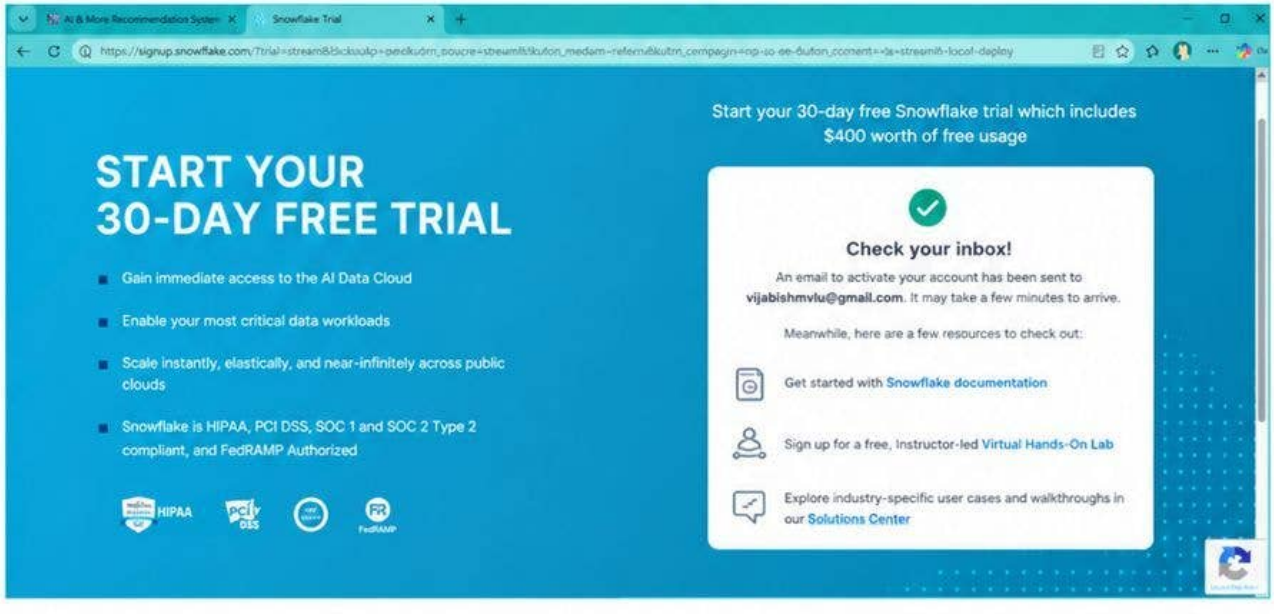

AI Movie Recommendation Syster
localhost:8501
Chat
Deploy
AI-Based Movie Recommendation System
An AI-based movie recommendation system that recommends similar movies using Content-Based Filtering, TF-IDF and Cosine Similarity.
A Group Project Made By Ishita & Subhashree
Select a Movie
Choose a movie you like:
Captain America The First Avenger
Number of recommendations
Recommend Movies
Recommended Movies
1. Captain America The Winter Soldier
Genre: Action Adventure Thriller
Similarity Score: 44.54%
2. Spider Man
Genre: Action Adventure Science Fiction
Similarity Score: 30.63%

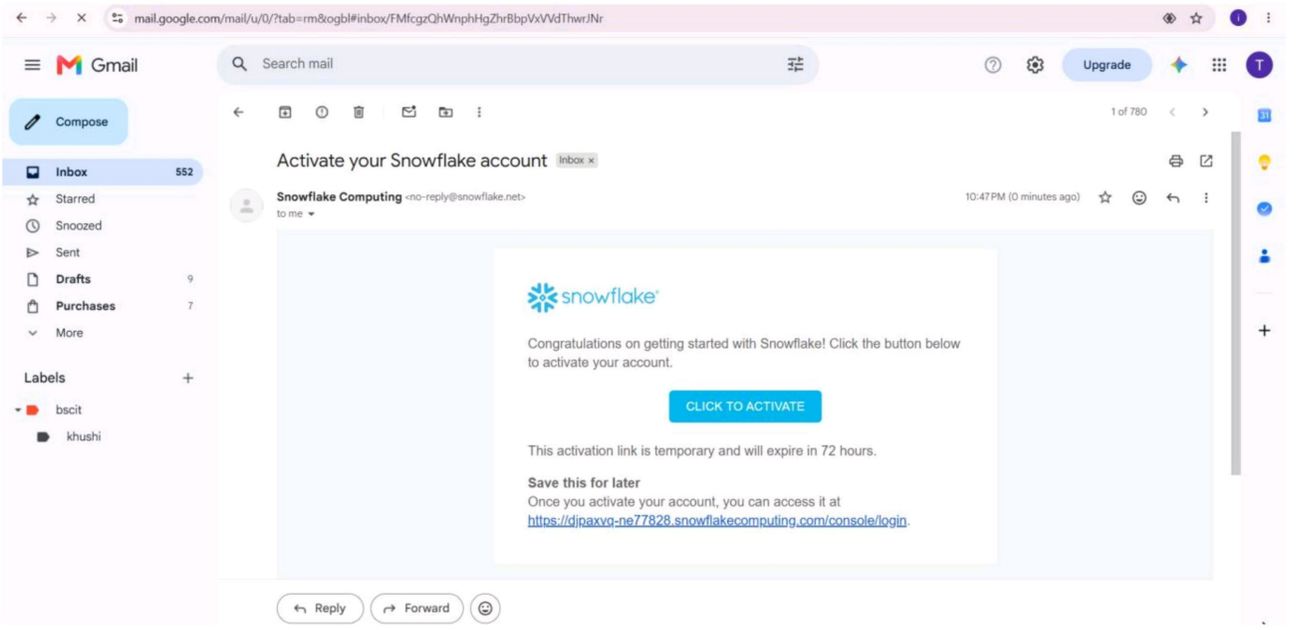

AI Movie Recommendation Syster
localhost:8501
Deploy
Genre: Action Science Fiction
Similarity Score: 21.82%
Show Recommendation Chart
Movie Dataset
View Movie Dataset
How AI Recommendation Works
1. The system stores movie information such as title, genre and description.
2. Movie information is converted into numerical features using TF-IDF Vectorization.
3. Cosine Similarity calculates how similar movies are to each other.
4. Movies with the highest similarity scores are selected as recommendations.
5. The recommended movies are displayed through the Streamlit interface.
About This Project
This project uses Content-Based Filtering to recommend movies based on their similarity.
TF-IDF is used to convert movie descriptions and genres into numerical features, while Cosine Similarity is used to measure similarity between movies.
This project is developed for educational purposes to demonstrate Artificial Intelligence and Machine Learning concepts.


AI Movie Recommendation Syster
localhost:8501
Chat
Deploy
Genre: Action Science Fiction
Similarity Score: 21.82%
Deploy this app using...
Streamlit Community Cloud
For community, always free
For personal hobbies and learning
Deploy unlimited public apps
Explore and learn from Streamlit's community and popular apps
Deploy now
Learn more
Snowflake
For enterprise
Enterprise-level security, support, and fully managed infrastructure
Deploy unlimited private apps with role-based sharing
Integrate with Snowflake's full data stack
Start trial
Learn more
Other platforms
For custom deployment
Deploy on your own hardware or cloud service
Set up and maintain your own authentication, resources, and costs
Learn more
Show Recommendation Chart
Movie Dataset
View Movie Dataset
How AI Recommendation Works
1. The system stores movie information such as title, genre and description.
2. Movie information is converted into numerical features using TF-IDF Vectorization.
3. Cosine Similarity calculates how similar movies are to each other.
4. Movies with the highest similarity scores are selected as recommendations.
5. The recommended movies are displayed through the Streamlit interface.


Start your 30-day free Snowflake trial which includes $400 worth of free usage
START YOUR 30-DAY FREE TRIAL
Gain immediate access to the AI Data Cloud
Enable your most critical data workloads
Scale instantly, elastically, and near-infinitely across public clouds
Snowflake is HIPAA, PCI DSS, SOC 1 and SOC 2 Type 2 compliant, and FedRAMP Authorized
HIPAA
Create a Snowflake account
1 / 3
Already have an account? Sign In
First name
Vijay
Last name
Bisht
Work email
vijabishmvlu@gmail.com
Why are you signing up?
Personal learning and development
Continue
Country: India


Start your 30-day free Snowflake trial which includes $400 worth of free usage
START YOUR 30-DAY FREE TRIAL
Gain immediate access to the AI Data Cloud
Enable your most critical data workloads
Scale instantly, elastically, and near-infinitely across public clouds
Snowflake is HIPAA, PCI DSS, SOC 1 and SOC 2 Type 2 compliant, and FedRAMP Authorized
HIPAA
Now, let's set up your account
2 / 3
Company name
Job title
Student
Choose your Snowflake edition
Learn more
Standard
Choose your cloud provider
Snowflake runs on top of the cloud platforms you are already working with.
Microsoft Azure
Amazon Web Services
Google Cloud Platform
Region
Asia Pacific (Thailand)
I have read and agree to the Snowflake Self-Service On Demand Terms


Start your 30-day free Snowflake trial which includes $400 worth of free usage
START YOUR 30-DAY FREE TRIAL
Gain immediate access to the AI Data Cloud
Enable your most critical data workloads
Scale instantly, elastically, and near-infinitely across public clouds
Snowflake is HIPAA, PCI DSS, SOC 1 and SOC 2 Type 2 compliant, and FedRAMP Authorized
HIPAA
Check your inbox!
An email to activate your account has been sent to vijabishmvlu@gmail.com. It may take a few minutes to arrive.
Meanwhile, here are a few resources to check out:
Get started with Snowflake documentation
Sign up for a free, Instructor-led Virtual Hands-On Lab
Explore industry-specific user cases and walkthroughs in our Solutions Center



RESULT

The Smart Movie Recommendation System Using Machine Learning was successfully developed and implemented using Python, Pandas, Scikit-learn, and Streamlit. The system analyzes movie genres and descriptions using TF-IDF Vectorization and calculates similarity between movies using Cosine Similarity.

When a user selects a movie, the system successfully identifies and displays the most similar movies along with their genre and similarity scores. The system also provides a movie similarity chart for better visualization and allows users to view the complete movie dataset.

The final result demonstrates that Content-Based Filtering can be effectively used to provide relevant movie recommendations based on the content of a selected movie. The project successfully demonstrates the practical application of Machine Learning and AI concepts in recommendation systems.

ADVANTAGES

1. Easy to Use – The Streamlit interface provides a simple and user-friendly way to select a movie and get recommendations.
2. Relevant Recommendations – The system recommends movies based on the similarity of their genre and description.
3. Saves Time – Users can quickly find similar movies without manually searching through a large collection.
4. Content-Based Recommendations – Recommendations are generated based on movie content, so information about other users is not required.
5. Uses Machine Learning Techniques – TF-IDF and Cosine Similarity provide an effective method for comparing movie content.
6. Similarity Scores – The system displays similarity scores, helping users understand how closely the recommended movies match the selected movie.
7. Visual Representation – The similarity chart provides an easy-to-understand comparison of recommended movies.
8. Simple and Low-Cost Implementation – The system can be developed using freely available Python libraries without requiring an external recommendation API.

LIMITATIONS

1. Limited Dataset – The system uses a small sample dataset containing only 20 movies, so recommendations may be limited.
2. Limited Features – Recommendations are based mainly on movie genre and description and do not consider factors such as ratings, actors, directors, or user preferences.
3. No User History – The system does not learn from the user's previous viewing history or personal preferences.
4. Content Dependency – The quality of recommendations depends on the information available in the movie descriptions and genres.
5. No Real-Time Data – The system does not use live movie databases or continuously updated movie information.
6. Cold-Start Limitation – New movies with limited or incomplete information may not receive highly accurate recommendations.
7. No Collaborative Filtering – The system does not compare the user's preferences with the preferences of other users.
8. Educational Prototype – The system is mainly developed to demonstrate Machine Learning and recommendation-system concepts and may require a larger, real-world dataset for practical deployment.

APPLICATIONS

The Smart Movie Recommendation System Using Machine Learning can be used in the following applications:

- Streaming Platforms – Can recommend movies similar to the movie selected or watched by the user.
- Online Movie Websites – Helps users discover movies that match their interests.
- Personalized Entertainment – Provides customized movie suggestions based on movie content and user selection.
- Movie Search Systems – Helps users find similar movies quickly from a large collection.
- Entertainment Applications – Can be integrated into mobile or web-based entertainment applications.
- Digital Libraries – Can help organize and recommend movies from a large digital movie collection.
- Educational Projects – Useful for demonstrating Machine Learning, Natural Language Processing, TF-IDF, and Cosine Similarity concepts.
- Future Recommendation Systems – Can be extended with ratings, user history, and collaborative filtering to build more advanced recommendation platforms

CONCLUSION

The Smart Movie Recommendation System Using Machine Learning was successfully developed using Python, Pandas, Scikit-learn, and Streamlit. The system uses Content-Based Filtering to recommend movies based on their genre and description.

TF-IDF Vectorization is used to convert movie information into numerical features, while Cosine Similarity is used to measure the similarity between movies. Based on the similarity scores, the system successfully identifies and recommends the most similar movies selected by the user.

The Streamlit interface makes the system simple and user-friendly by allowing users to select a movie, choose the number of recommendations, and view the recommended movies with their similarity scores and charts.

Overall, the project demonstrates the practical application of Machine Learning, Natural Language Processing, and recommendation techniques to create an intelligent movie recommendation system.

FUTURE SCOPES

The Smart Movie Recommendation System Using Machine Learning can be further improved and expanded in the following ways:

1. Larger Movie Dataset – Use a larger and regularly updated movie dataset containing thousands of movies.
2. User Preferences – Allow users to provide their preferred genres, languages, actors, or directors for more personalized recommendations.
3. User Ratings and Reviews – Include movie ratings and reviews to improve the quality of recommendations.
4. Collaborative Filtering – Implement collaborative filtering to recommend movies based on the preferences and viewing patterns of similar users.
5. Hybrid Recommendation System – Combine content-based filtering and collaborative filtering to provide more accurate recommendations.
6. User History – Store previously watched or liked movies and use this information to generate personalized recommendations.
7. Advanced Machine Learning – Use advanced ML or deep learning techniques to improve recommendation accuracy.
8. Real-Time Movie Database – Connect the system with an online movie database to provide updated information about newly released movies.
9. Mobile Application – Develop a mobile version so users can access movie recommendations from smartphones.
10. Improved User Interface – Add movie posters, trailers, ratings, filters, and search features to make the system more interactive and attractive.

REFERENCES

1. Python Documentation – Python programming language and libraries.
2. Pandas Documentation – Used for creating and processing the movie dataset.
3. Scikit-learn Documentation – Used for TF-IDF Vectorization and Cosine Similarity.
4. Streamlit Documentation – Used for developing the interactive web application.
5. Machine Learning and Recommendation System Study Materials – Used to understand Content-Based Filtering and recommendation techniques.
6. Project-Created Movie Dataset – Sample dataset containing movie titles, genres, and descriptions used for educational implementation.

GITHUB

https://github.com/ishipatelmvlu-sys/AI/blob/main/appp.py